

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1. Анализ предметной области	6
1.1 Характеристика предметной области	6
1.2 Роли пользователей системы.....	7
1.3 Основные бизнес-процессы	8
1.4 Ограничения учебной модели	10
1.5 Постановка задачи	11
2. Проектирование базы данных	12
2.1 Требования к хранению данных	12
2.2 Описание сущностей	13
2.3 Связи между сущностями	14
2.4 Реализация базы данных в SQLite	15
2.5 Схема базы данных.....	16
3. Разработка сайта	17
3.1 Общая архитектура проекта	17
3.2 Пользовательский интерфейс и навигация	18
3.3 Авторизация, регистрация и управление сессией.....	19
3.4 Пользовательские страницы	20
3.5 Торговая страница и графики	23
3.6 Клиентская логика JavaScript	24
3.7 Backend/API и работа с SQLite.....	25
3.8 Административная панель и симуляция	26
3.9 Оформление, футер и документы	27
3.10 Алгоритмы работы программы	28
4. Тестирование	29
4.1 Методика тестирования	29
4.2 Направления проверки	30
4.3 Тестовые сценарии.....	30
4.4 Анализ результатов тестирования.....	33
ЗАКЛЮЧЕНИЕ	34
СПИСОК СОКРАЩЕНИЙ	37
СПИСОК ИСТОЧНИКОВ	37
ПРИЛОЖЕНИЕ А ТЕКСТ ПРОГРАММЫ	
ПРИЛОЖЕНИЕ Б БЛОК-СХЕМА ПРОГРАММЫ	

					КП.АС576.220249 – 05 81 00			
Изм.	Лист	№ докум.	Подпись	Дата				
Разраб.					Разработка многостраничного веб-сайта учебной криптобиржи с адаптивной версткой. Пояснительная записка	Лит.	Лист	Листов
Провер.	Демидович А.Г.							
Н. Контр.						УО «БрГТУ»		
Утверд.								

ВВЕДЕНИЕ

В настоящее время веб-приложения широко применяются для автоматизации учебных, коммерческих и информационных процессов. Одной из распространенных предметных областей являются электронные торговые платформы, позволяющие пользователю просматривать активы, выполнять операции покупки и продажи, анализировать состояние портфеля и историю действий.

Криптовалютные сервисы являются наглядным примером информационной системы, в которой одновременно используются учетные записи пользователей, справочники активов, операции с балансами, история транзакций, графики изменения цен и административные функции. Даже в учебной реализации такая система требует продуманной структуры данных и согласованной логики обработки пользовательских действий.

Актуальность темы курсового проекта связана с тем, что разработка торгового веб-сайта позволяет показать сразу несколько важных направлений веб-разработки: создание многостраничного интерфейса, регистрацию и авторизацию пользователей, работу с базой данных, обработку форм, проверку корректности операций, получение данных из внешних источников и отображение динамических графиков.

Целью курсового проекта является разработка сайта CryptoTrade для учебной симуляции торговли криптовалютой. Сайт должен позволять пользователю зарегистрироваться, войти в систему, просматривать рынок криптовалют, выполнять учебные операции покупки и продажи, анализировать портфель и историю операций, а администратору - управлять активами и просматривать общую статистику.

Для достижения поставленной цели необходимо решить следующие задачи: выполнить анализ предметной области; определить роли пользователей; спроектировать логическую и физическую модель базы данных; реализовать хранение данных в SQLite; разработать Python backend/API; создать пользовательские страницы сайта; реализовать торговую форму, портфель, историю, административную панель, графики и симуляцию активности; провести тестирование основных сценариев.

Объектом разработки является веб-сайт CryptoTrade. Предметом разработки является организация пользовательского интерфейса, логики авторизации, торговли, хранения данных, получения рыночных свечей, построения графиков и административного управления рынком активов.

Практическая часть проекта реализована как локальное веб-приложение. Пользовательский интерфейс выполнен средствами HTML, CSS и JavaScript. Серверная часть реализована на Python и предоставляет API для работы с SQLite-базой cryptotrade.db. Такой подход позволяет показать не только внешний вид сайта, но и реальное постоянное хранение пользователей, кошельков, транзакций, истории цен и настроек симуляции.

					КП.AC576.220249 – 05 81 00	Лист
						4
Изм.	Лист	№ докум.	Подпись	Дата		

В рамках проекта используются учебные денежные средства, поэтому сайт не выполняет настоящие финансовые операции и не подключается к реальным криптовалютным кошелькам. При этом для большей наглядности добавлены открытые источники рыночных данных, свечные графики, fallback-история цен, политика конфиденциальности и пользовательское соглашение.

Пояснительная записка включает анализ предметной области, проектирование базы данных, описание разработки сайта, тестирование, заключение, список сокращений, список источников и приложения. В приложениях предполагается разместить описание файлов проекта, блок-схему алгоритма работы программы, схему базы данных и экранные формы сайта.

1. Анализ предметной области

1.1 Характеристика предметной области

Криптовалютная биржа представляет собой информационную систему, предназначенную для организации операций с цифровыми активами. Пользователь такой системы получает сведения о доступных криптовалютах, анализирует текущие цены, выбирает актив и выполняет операции покупки или продажи. Результаты операций отражаются в состоянии счета, портфеля и истории действий пользователя.

Криптовалютные торговые платформы относятся к классу финансово-информационных систем. Их работа связана с обработкой учетных записей пользователей, хранением информации об активах, расчетом стоимости операций и ведением журнала выполненных транзакций. Даже в учебной модели такая система должна поддерживать согласованность данных: при покупке уменьшается денежный баланс и увеличивается количество монет, а при продаже выполняется обратная операция.

В реальных криптовалютных биржах дополнительно используются платежные шлюзы, внешние кошельки, комиссии, механизмы подтверждения личности, биржевые стаканы, ордера разных типов и средства защиты от несанкционированного доступа. В рамках курсового проекта эти механизмы упрощены, так как основная задача состоит не в создании промышленной биржи, а в демонстрации принципов разработки веб-сайта с авторизацией, регистрацией, логической базой данных, пользовательской частью и административной панелью.

Предметная область проекта выбрана таким образом, чтобы показать работу с несколькими связанными сущностями данных. Пользователь взаимодействует с рынком активов, на рынке представлены криптовалюты, каждая операция фиксируется как транзакция, а портфель пользователя формируется на основании кошельков и актуальных цен. Это позволяет построить SQLite-модель данных, содержащую больше пяти связанных таблиц, что соответствует требованиям курсового проекта.

Система CryptoTrade является учебной симуляцией. Пользователь не работает с реальными денежными средствами, а получает демонстрационный баланс в долларах США. Такой подход позволяет безопасно проверить основные функции торговой платформы: регистрацию, вход, выбор актива, покупку, продажу, просмотр портфеля, анализ истории операций и работу с рыночными графиками.

Для приближения учебной модели к реальным торговым платформам в проект добавлены автоматическое обновление курсов, свечные данные, расширенный график на базе TradingView Lightweight Charts, политика конфиденциальности и пользовательское соглашение. При недоступности внешних источников сайт использует локальную историю цен из базы данных.

Общая схема предметной области должна отражать связь между пользователем, рынком активов, торговой операцией, портфелем, историей и администратором. Данная схема будет размещена после первой ссылки на нее, как показано на рисунке 1.1.



Рисунок 1.1 - Общая схема предметной области

1.2 Роли пользователей системы

В проектируемой системе выделяются две пользовательские роли: трейдер и администратор, а также служебная роль simulator для данных, созданных режимом симуляции. Разделение ролей необходимо для разграничения пользовательских и управляющих функций. Обычный пользователь должен выполнять только операции, связанные с личным учебным счетом, а администратор должен иметь возможность управлять рынком и контролировать действия пользователей.

Трейдер является основным пользователем сайта. Он проходит регистрацию, выполняет вход в систему, получает демонстрационный баланс, просматривает рынок криптовалют, выбирает актив и совершает учебные операции покупки или продажи. После выполнения операций трейдер может просматривать собственный портфель и историю своих сделок.

Администратор отвечает за управление учебной биржей. В отличие от трейдера, он имеет доступ к административной панели. В ней отображается общая статистика: количество пользователей, количество сделок и оборот операций. Администратор может обновлять курс существующей криптовалюты, выбирать ручной или автоматический режим цены, добавлять новые активы, просматривать журнал операций всех пользователей и управлять симуляцией активности.

Особенность административной роли заключается в том, что администратор работает не только со своими данными, а с данными системы в целом. Например, обычный пользователь видит только свою историю операций, а администратор видит об-

ций журнал сделок с указанием пользователя, типа операции, актива, количества, цены и суммы.

Роль simulator не предназначена для ручного входа на сайт. Она назначается пользователям, созданным автоматической симуляцией, чтобы отделить учебную нагрузку от реальных зарегистрированных пользователей. При очистке симуляции удаляются только записи с признаком isSimulated.

Роли пользователей системы представлены в таблице 1.1.

Таблица 1.1 - Роли пользователей системы

Роль	Основные функции	Ограничения
Треjder	Регистрация, вход, просмотр рынка, покупка и продажа активов, просмотр портфеля и истории	Не имеет доступа к административному управлению и операциям других пользователей
Администратор	Просмотр статистики, управление курсами, добавление активов, просмотр операций всех пользователей	Не предназначен для обычной торговли как основной сценарий работы
Simulator	Создание учебных пользователей и сделок режимом симуляции	Служебная роль, не используется для ручной авторизации

1.3 Основные бизнес-процессы

Бизнес-процесс в рамках данного курсового проекта понимается как последовательность действий пользователя или администратора, приводящая к изменению состояния системы. Для учебной криптовалютной биржи основными процессами являются регистрация, авторизация, просмотр рынка, выполнение сделки, просмотр портфеля, просмотр истории и административное управление активами.

Процесс регистрации начинается с ввода имени, электронной почты и пароля. После проверки корректности данных создается новая учетная запись пользователя. Новому пользователю назначается роль трейдера и демонстрационный баланс. После успешной регистрации пользователь автоматически переходит в личный кабинет.

Процесс авторизации выполняется при вводе email и пароля. Система проверяет наличие пользователя и соответствие пароля. Если данные корректны, создается текущая сессия пользователя. Если авторизуется администратор, он направляется в административную панель, а если обычный трейдер - на страницу обзора портфеля.

Процесс покупки актива включает выбор криптовалюты, ввод количества, расчет стоимости операции и проверку доступного USD-баланса. Если денежных средств достаточно, система уменьшает USD-баланс, увеличивает количество монет в кошель-

ке пользователя и фиксирует транзакцию. Если средств недостаточно, пользователь получает сообщение об ошибке.

Процесс продажи актива аналогичен покупке, но проверяется не денежный баланс, а количество монет в кошельке. При успешной продаже количество монет уменьшается, а USD-баланс пользователя увеличивается на сумму сделки. Сведения о продаже сохраняются в таблице транзакций.

Административные процессы включают изменение курса актива, добавление новой монеты, выбор источника цены и запуск симуляции активности. При изменении курса обновляется текущая цена и пополняется история цен. При добавлении монеты администратор указывает символ, название, начальную цену, уровень риска, описание и режим получения цены.

Процесс получения рыночных данных выполняется через Python API. Сайт запрашивает свечи по выбранной торговой паре, сначала используется Binance Spot, затем Coinbase Exchange как запасной источник. Если внешние источники недоступны, графики строятся по локальной истории цен из SQLite.

Процесс симуляции активности управляется вручную из админ-панели. Администратор задает количество симулируемых пользователей и сделок в минуту. Система не догоняет пропущенное время после закрытия сервера, поэтому после нового запуска генерация продолжается только с текущего момента.

Основные процессы системы приведены в таблице 1.2.

Таблица 1.2 - Основные бизнес-процессы системы

Процесс	Участник	Результат
Регистрация	Треjder	Создание учетной записи и демонстрационного баланса
Авторизация	Треjder, администратор	Создание пользовательской сессии и переход в соответствующий раздел
Покупка актива	Треjder	Уменьшение USD-баланса, увеличение количества монет, создание операции
Продажа актива	Треjder	Уменьшение количества монет, увеличение USD-баланса, создание операции
Просмотр портфеля	Треjder	Получение сведений о денежных средствах и криптоактивах
Изменение курса	Администратор	Обновление цены актива и истории цен
Автообновление курсов	Система, администратор	Получение цены из открытого источника и сохранение результата в SQLite
Построение графика	Треjder	Получение свечей через API и отображение графика на странице торговли
Симуляция активности	Администратор	Создание отделенных симулируемых пользователей и операций
Просмотр операций	Администратор	Контроль общего журнала пользовательских сделок

1.4 Ограничения учебной модели

Проект CryptoTrade является учебной моделью, поэтому в нем сознательно введены ограничения. Система может получать публичные рыночные данные из открытых API, но не обрабатывает банковские карты, не использует реальные кошельки и не выполняет настоящие финансовые операции. Все действия выполняются с демонстрационным балансом.

Данные хранятся в локальной SQLite-базе `site/cryptotrade.db`. Такое решение выбрано для учебного проекта, потому что SQLite не требует установки отдельного сервера СУБД, но при этом предоставляет реальные таблицы, внешние ключи и постоянное хранение между запусками.

SQLite является файловой СУБД и подходит для локального учебного проекта. Для промышленной эксплуатации криптовалютной биржи потребовалась бы серверная база данных, например PostgreSQL или MySQL, а также полноценная серверная авторизация, хеширование паролей и резервное копирование.

Еще одним ограничением является отсутствие реального механизма безопасности. Пароли в учебной модели сохраняются в открытом виде, потому что целью проекта является демонстрация логики сайта, а не разработка промышленной системы защиты. В реальной системе потребовалось бы хэширование паролей, серверная авторизация, защита от подбора паролей и разграничение доступа на уровне серверной части.

Основные ограничения учебной модели представлены в таблице 1.3.

Таблица 1.3 - Ограничения учебной модели

Ограничение	Причина использования	Возможное развитие
Демонстрационный баланс	Безопасная имитация торговых операций	Подключение платежной системы или тестовой серверной среды
Хранение в SQLite	Реальная файловая база данных без установки отдельного сервера СУБД	Использование PostgreSQL или MySQL
Зависимость автообновления от внешней сети	Получение публичных рыночных данных из открытых источников	Кэширование и несколько резервных поставщиков данных
Упрощенная авторизация	Демонстрация принципа входа и регистрации	Серверная авторизация и хэширование паролей
Отсутствие реальных кошельков	Исключение финансовых рисков	Интеграция с тестовой блокчейн-сетью

1.5 Постановка задачи

На основании анализа предметной области необходимо разработать сайт, который обеспечивает работу с учебным торговым счетом и демонстрирует основные функции криптовалютной биржи. Сайт должен быть понятен пользователю, иметь несколько связанных страниц, локальный Python API и обеспечивать сохранение данных в SQLite между посещениями.

В практической части требуется реализовать регистрацию и авторизацию пользователей. Пользователь должен иметь возможность создать учетную запись, войти в систему и выйти из нее. Для проверки работы должны быть предусмотрены демонстрационные учетные записи трейдера и администратора.

Пользовательская часть должна включать просмотр рынка криптовалют, выполнение операций покупки и продажи, отображение текущего портфеля, просмотр истории операций и расширенный график на базе свечных данных. Все операции должны сопровождаться проверкой доступного баланса или количества монет. При ошибке система должна выводить понятное сообщение.

Административная часть должна предоставлять функции управления учебной биржей. Администратор должен видеть статистику системы, общий журнал операций пользователей, список активов, форму обновления курса, форму добавления новой монеты, настройки источника цены и управление симуляцией активности. Доступ к административному разделу должен быть ограничен ролью администратора.

Для выполнения требований по базе данных необходимо спроектировать логическую модель не менее чем из пяти связанных сущностей и реализовать ее в SQLite. В текущем проекте используется семь таблиц: роли, пользователи, валюты, кошельки, транзакции, история цен и системные настройки. Такая структура позволяет показать связи между пользователями, активами, торговыми операциями и режимом симуляции.

Итогом выполнения задачи должен стать работоспособный сайт с локальным Python-сервером, SQLite-базой, API, страницами политики конфиденциальности и пользовательского соглашения, а также пояснительная записка, содержащая анализ предметной области, проектирование данных, описание разработки, алгоритмы работы программы, блок-схему, тестирование, список сокращений, список источников и приложения.

2. Проектирование базы данных

2.1 Требования к хранению данных

База данных сайта CryptoTrade должна хранить сведения о пользователях, ролях, криптовалютных активах, кошельках пользователей, завершенных операциях, истории изменения курсов и системных настройках симуляции. Эти данные необходимы для авторизации, разграничения прав доступа, расчета портфеля, выполнения сделок, построения локальной fallback-истории цен и вывода отчетной информации в административной панели.

Для учебного проекта используется логическая модель базы данных. Она описывается как набор связанных сущностей, аналогичных таблицам реляционной базы данных. В программной реализации данные сохраняются в SQLite-базе cryptotrade.db. Сайт обращается к базе через Python API в файле backend.py, что позволяет показать основные операции с данными: создание записей, чтение, изменение балансов и формирование истории операций.

К хранению данных предъявляются следующие требования: каждая запись должна иметь уникальный идентификатор; пользователь должен относиться к одной роли; торговая операция должна быть связана с пользователем и выбранной криптовалютой; кошелек должен хранить количество конкретной монеты у конкретного пользователя; история операций должна сохраняться после обновления страницы; симулируемые данные должны отделяться от реальных; администратор должен иметь возможность просматривать операции всех пользователей.

Дополнительно система должна проверять корректность данных перед сохранением. При регистрации проверяется уникальность email и минимальная длина пароля. При покупке проверяется достаточность USD-баланса, при продаже - достаточность количества монет в кошельке. Цена актива и количество в сделке должны быть больше нуля. Эти проверки защищают логическую целостность учебной базы данных.

В SQLite создаются таблицы roles, users, currencies, wallets, transactions, price_history и system_settings. Таблицы связаны внешними ключами. В localStorage остаются только настройки браузера: тема интерфейса, текущая локальная сессия и служебная информация о последнем запросе курса.

2.2 Описание сущностей

В логической модели выделено шесть основных сущностей предметной области и служебная сущность настроек. Их назначение и состав полей приведены в таблице 2.1. Названия полей в таблице приведены в учебном SQL-подобном виде. В JavaScript-реализации часть полей записана в стиле camelCase: например, balance_usd соответствует balanceUsd, user_id соответствует userId, currency_id соответствует currencyId, price_mode соответствует priceMode, market_symbol соответствует marketSymbol.

Таблица 2.1 - Сущности базы данных и их поля

Сущность	Основные поля	Назначение
roles	id, code, title	Справочник ролей. В системе используются роли trader, admin и simulator. Роль определяет доступ к обычным страницам, административной панели или принадлежность к симуляции.
users	id, role_id, name, email, password, balance_usd, created_at, is_simulated	Пользователи сайта. Сущность хранит учетные данные, роль, имя пользователя, дату создания, учебный USD-баланс и признак симуляции.
currencies	id, symbol, name, price, color, risk, description, price_mode, market_symbol, source, last_sync_at, last_sync_status, last_sync_message	Криптовалютные активы рынка. Для каждого актива задаются символ, название, текущая цена, уровень риска, описание и источник обновления курса.
wallets	id, user_id, currency_id, amount, updated_at, is_simulated	Кошельки пользователей. Запись показывает, сколько конкретной криптовалюты принадлежит конкретному пользователю.
transactions	id, user_id, currency_id, side, quantity, price, total, status, created_at, is_simulated	Завершенные операции. Сущность используется для истории пользователя и общего журнала в административной панели.
price_history	id, currency_id, price, recorded_at, position	История изменения курсов. Используется для локальной динамики, fallback-графиков и хранения последних сохраненных цен.
system_settings	id, simulation_enabled, simulation_level, simulation_users_target, simulation_trades_per_minute, last_simulation_at.	Служебные настройки режима симуляции активности: включение, число симулируемых пользователей и частота сделок.

Сущность `roles` нужна для разграничения прав доступа. Обычный трейдер может выполнять операции с собственным счетом, смотреть рынок, портфель и личную историю. Администратор дополнительно получает доступ к статистике, списку всех операций пользователей, обновлению курсов и добавлению новых активов. Роль `simulator` используется для пользователей, созданных режимом симуляции активности.

Сущность `users` является центральной для пользовательской части сайта. При регистрации создается новая запись пользователя с ролью `trader` и стартовым балансом 10000 USD. При входе в систему данные пользователя сопоставляются с `email` и паролем, после чего идентификатор пользователя сохраняется в сессии.

Сущность `currencies` хранит рыночные активы. В начальном наборе используются BTC, ETH, SOL, ADA, BNB, XRP, DOGE, AVAX, DOT, LINK, LTC и TRX. Администратор может добавить новую монету, задать ручной курс или включить автоматическое обновление по биржевой паре. Цена актива применяется при расчете суммы сделки и стоимости портфеля. На странице торговли данные используются для построения линейного графика или японских свечей с индикатором SMA.

Сущность `wallets` связывает пользователя и криптовалюту. Если пользователь впервые покупает актив, запись кошелька создается автоматически. При покупке количество монет увеличивается, а при продаже уменьшается. Если количество становится близким к нулю, в интерфейсе такой актив не отображается в портфеле.

Сущность `transactions` фиксирует торговый процесс. Так как в учебной модели каждая операция выполняется сразу, отдельная таблица заявок не используется. В транзакции сохраняются тип операции `buy` или `sell`, количество, цена, сумма, дата и признак симуляции.

Сущность `system_settings` хранит состояние симуляции: включена ли она, сколько симулируемых пользователей должно быть создано, сколько сделок в минуту необходимо генерировать и какой дробный остаток сделок перенесен на следующий цикл. Это нужно для равномерной генерации без догоняющего выполнения после закрытия сервера.

2.3 Связи между сущностями

Связи между сущностями отражают, какие записи зависят друг от друга. Основные связи приведены в таблице 2.2.

					КП.АС576.220249 – 05 81 00	Лист
						14
Изм.	Лист	№ докум.	Подпись	Дата		

Таблица 2.2 - Связи между сущностями

Связь	Тип	Описание
users.role_id -> roles.id	многие к одному	Каждый пользователь имеет одну роль, а одна роль может быть назначена многим пользователям.
wallets.user_id -> users.id	многие к одному	У одного пользователя может быть несколько кошельков для разных криптовалют.
wallets.currency_id -> currencies.id	многие к одному	Одна криптовалюта может присутствовать в кошельках разных пользователей.
transactions.user_id -> users.id	многие к одному	История операций позволяет получить все сделки конкретного пользователя.
transactions.currency_id -> currencies.id	многие к одному	Операция связана с активом, по которому была выполнена сделка.
price_history.currency_id -> currencies.id	многие к одному	Для одного актива может храниться несколько исторических значений курса.
system_settings.id	одна запись настроек	Служебная запись main управляет параметрами симуляции активности.

На уровне интерфейса эти связи используются во всех основных разделах. На странице портфеля кошельки фильтруются по идентификатору текущего пользователя и связываются с таблицей активов для расчета стоимости. На странице истории транзакции фильтруются по пользователю и дополняются названием криптовалюты. В административной панели журнал транзакций выводится без фильтра по текущему пользователю, поэтому администратор видит операции всех зарегистрированных участников.

2.4 Реализация базы данных в SQLite

Физически данные размещаются в файле site/cryptotrade.db. База создается автоматически при запуске Python-сервера backend.py. Для работы используется стандартный модуль sqlite3, поэтому проект не требует установки внешней СУБД.

Пользователи хранятся отдельно от торговых данных. При входе система читает список пользователей из SQLite, ищет совпадение по email и паролю, а затем записывает текущую локальную сессию в браузер. Валюты, кошельки, транзакции, история цен и настройки симуляции хранятся в отдельных таблицах.

Python API предоставляет маршруты /api/state для чтения и сохранения состояния, /api/health для проверки работоспособности и /api/klines для получения рыночных

свечей. Файл базы данных не отдается как статический ресурс: запросы к .db блокируются сервером. Это отделяет интерфейс сайта от физического файла базы данных.

Для защиты уже созданных данных обновление состояния выполняется не полной перезаписью всех таблиц, а через добавление и обновление записей. При очистке симуляции удаляются только пользователи, кошельки и транзакции с признаком is_simulated. Реальные пользователи и реальные операции при этом сохраняются.

SQLite подходит для курсовой работы, потому что является полноценной файловой базой данных с таблицами и внешними ключами. Ограничением является локальный режим работы: для промышленной биржи потребовались бы серверная СУБД, хеширование паролей, миграции, журналирование и резервное копирование.

2.5 Схема базы данных

На рисунке 2.1 показана схема базы данных проекта CryptoTrade. В ней отражены основные таблицы SQLite и связи между ними. Центральными сущностями являются users и currencies, так как через них связаны кошельки, транзакции и история цен. Таблица system_settings является служебной и хранит единственную запись настроек симуляции.

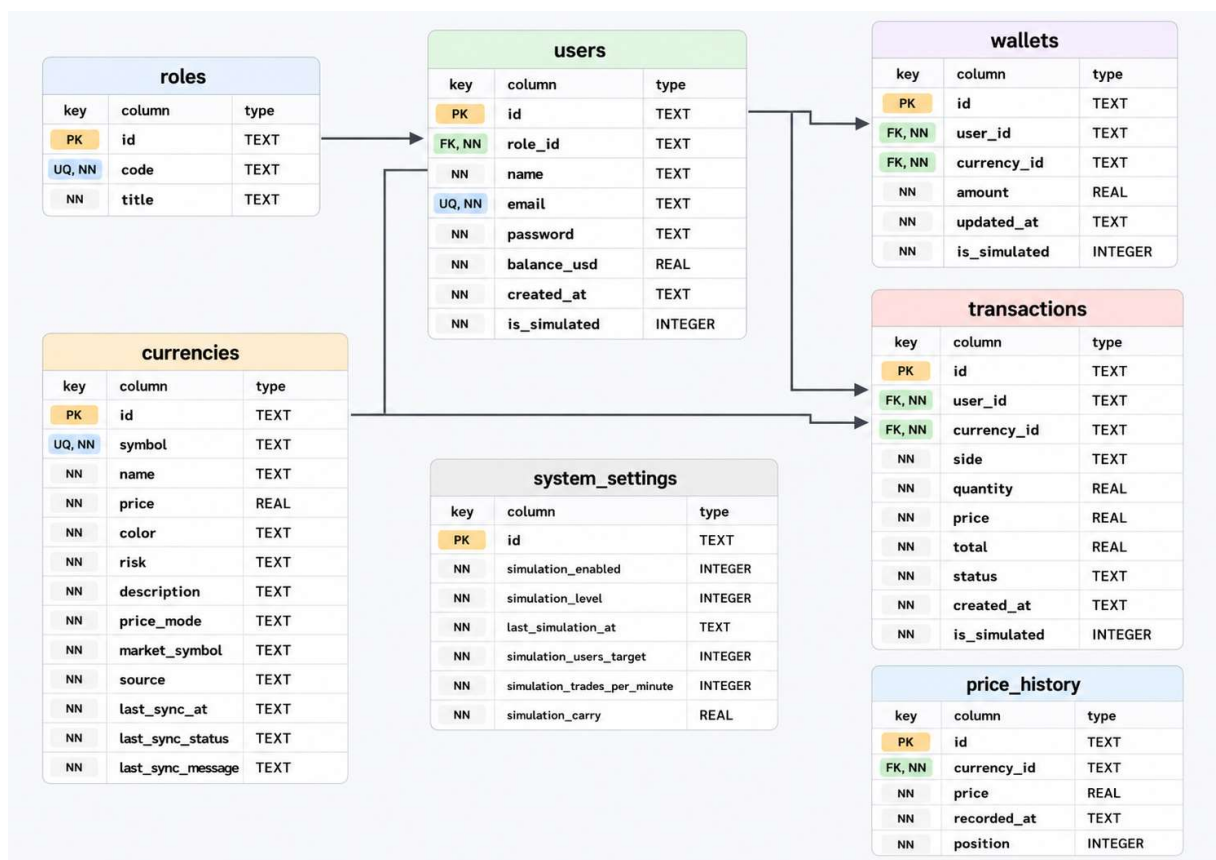


Рисунок 2.1 - Схема базы данных

3. Разработка сайта

3.1 Общая архитектура проекта

Практическая часть курсового проекта реализована как локальное веб-приложение CryptoTrade. Пользователь работает с HTML-страницами в браузере, клиентская логика выполняется на JavaScript, визуальное оформление задается CSS, а постоянное хранение данных выполняется в SQLite-базе cryptotrade.db через Python backend/API.

Проект запускается через файл start_cryptotrade.bat из корня проекта или напрямую командой python backend.py из папки site. После запуска Python-сервер открывает локальный адрес <http://127.0.0.1:8000/index.html>, отдает статические файлы сайта и обрабатывает API-запросы. Такой подход выбран потому, что браузер не может напрямую читать и изменять SQLite-файл, а серверная прослойка позволяет безопаснее отделить интерфейс от физической базы данных.

Архитектура проекта разделена на три уровня. Первый уровень - представление: HTML-страницы index.html, dashboard.html, market.html, trade.html, portfolio.html, history.html, admin.html, privacy.html и terms.html. Второй уровень - клиентская логика в assets/js/app.js, где выполняются авторизация, расчеты, обновление интерфейса, торговые операции, работа с графиками и обращение к API. Третий уровень - backend.py, который инициализирует SQLite, отдает состояние приложения и получает свечные данные из внешних источников.

Основной обмен между интерфейсом и сервером выполняется в формате JSON. Клиент загружает состояние через /api/state, после изменений отправляет обновленное состояние POST-запросом на /api/state, а для графиков запрашивает свечи через /api/klines. Благодаря этому страницы остаются обычными HTML-документами, но данные сохраняются не в localStorage как в ранней версии, а в полноценной файловой базе данных SQLite.

Структура файлов проекта приведена в таблице 3.1.

Таблица 3.1 - Структура файлов проекта

Файл	Назначение
site/index.html	Страница входа и регистрации
site/dashboard.html	Обзор портфеля пользователя
site/market.html	Рынок криптовалют и лента активности
site/trade.html	Покупка, продажа и расширенный график
site/portfolio.html	Портфель пользователя и распределение активов
site/history.html	История операций текущего пользователя
site/admin.html	Административная панель

Продолжение таблицы 3.1

Файл	Назначение
site/privacy.html	Политика конфиденциальности
site/terms.html	Пользовательское соглашение
site/assets/css/style.css	Стили сайта, темная тема, адаптивная верстка
site/assets/js/app.js	Клиентская логика, торговля, графики, API-запросы
site/backend.py	Python API, статический сервер и работа с SQLite
site/cryptotrade.db	SQLite-база данных, создается и обновляется сервером
site/README.md	Краткая инструкция по запуску и описанию проекта
start_cryptotrade.bat	Запуск локального сервера и сайта из корня проекта

3.2 Пользовательский интерфейс и навигация

Интерфейс сайта построен как набор связанных страниц с общей навигацией. После входа пользователь видит верхнее меню: Обзор, Рынок, Торговля, Портфель, История. Для администратора дополнительно показывается пункт Админ. В правой части навигации отображается имя пользователя и кнопка выхода.

На всех основных страницах добавлен футер с названием проекта и ссылками на политику конфиденциальности и пользовательское соглашение. Наличие этих страниц делает учебный сайт более похожим на реальный сервис: пользователь может увидеть, какие данные используются, где они хранятся и почему операции не являются настоящими финансовыми сделками.

Визуальное оформление выполнено в темной теме. Для предотвращения белой вспышки при загрузке тема применяется в самом начале загрузки страницы. Переключение темы сохраняется в браузере как локальная настройка и не относится к предметной базе данных.

Для связывания HTML и JavaScript используются data-атрибуты. Например, data-page определяет текущую страницу, data-login-form и data-register-form используются для форм входа и регистрации, data-trade-form - для торговой формы, data-admin-price-form - для формы обновления курса. Такой подход позволяет не привязывать JavaScript к случайным CSS-классам и делает структуру интерфейса более понятной.

Страницы сайта приведены в таблице 3.2.

Таблица 3.2 - Страницы сайта

Страница	Основные элементы	Назначение
index.html	Форма входа, форма регистрации, демо-рынок, ссылки на документы	Вход пользователя, регистрация трейдера и переход в личный раздел
dashboard.html	Метрики портфеля, график BTC, последние сделки, лента рынка	Краткий обзор состояния пользователя после входа
market.html	Таблица активов, мини-графики, лента активности	Просмотр доступных криптовалют, цен, риска и динамики
trade.html	Выбор монеты, расчет суммы, покупка/продажа, TradingView-график	Выполнение учебных торговых операций
portfolio.html	Итоговая стоимость, USD-баланс, карточки активов	Отображение текущих средств пользователя
history.html	Таблица операций пользователя	Просмотр личной истории покупок и продаж
admin.html	Метрики, управление курсами, добавление монет, симуляция, журнал операций	Административное управление учебной биржей
privacy.html	Текст политики конфиденциальности	Описание учебного характера данных и хранения
terms.html	Текст пользовательского соглашения	Фиксация правил использования сайта

3.3 Авторизация, регистрация и управление сессией

Страница index.html содержит два режима: вход и регистрацию. По умолчанию отображается форма входа, а форма регистрации открывается отдельной кнопкой. Такой интерфейс уменьшает визуальную перегрузку страницы и позволяет пользователю выполнить нужное действие без перехода на отдельную страницу.

При регистрации пользователь вводит имя, email и пароль. JavaScript проверяет длину имени, наличие email, минимальную длину пароля и отсутствие пользователя с таким email. После успешной регистрации создается запись пользователя с ролью trader и стартовым балансом 10000 USD. Затем пользователь автоматически авторизуется и переходит на dashboard.html.

При входе функция authenticate сопоставляет email и пароль с данными пользователей, полученными через API из SQLite. Если данные неверны, выводится сообщение об ошибке. Если вход успешен, идентификатор пользователя сохраняется в ло-

кальной сессии браузера, после чего выполняется переход: обычный трейдер открывает страницу обзора, администратор - административную панель.

Проверка доступа выполняется при открытии каждой защищенной страницы. Если пользователь не вошел в систему, он перенаправляется на index.html. Если обычный трейдер пытается открыть admin.html, он возвращается на dashboard.html. Благодаря этому административные функции скрыты не только визуально, но и логически ограничены на уровне клиентского сценария.

УЧЕБНАЯ ТОРГОВАЯ СРЕДА

Вход в симулятор
криптовалютной биржи

Email

student@cryptotrade.local

Пароль

student123

Войти

Демо-доступ

Трейдер: student@cryptotrade.local / student123

Администратор: admin@cryptotrade.local / admin123

Зарегистрироваться

НОВЫЙ АККАУНТ

Регистрация трейдера

Имя

Ваше имя

Email

name@example.com

Пароль

Минимум 6 символов

Создать аккаунт

Каждый новый пользователь получает демо-баланс \$10 000.00.

Создавая аккаунт, пользователь принимает **соглашение** и **политику конфиденциальности**.

Уже есть аккаунт

Рисунок 3.1 – Макет страницы входа и регистрации

3.4 Пользовательские страницы

Страница dashboard.html является личным обзором пользователя. На ней отображаются общая стоимость портфеля, доступный USD-баланс и количество активов в портфеле. Дополнительно выводятся график динамики BTC, последние операции пользователя, краткая лента рынка и карточки популярных активов. Эта страница нужна как стартовая точка после входа. Макет такой страницы представлен на рисунке 3.2.

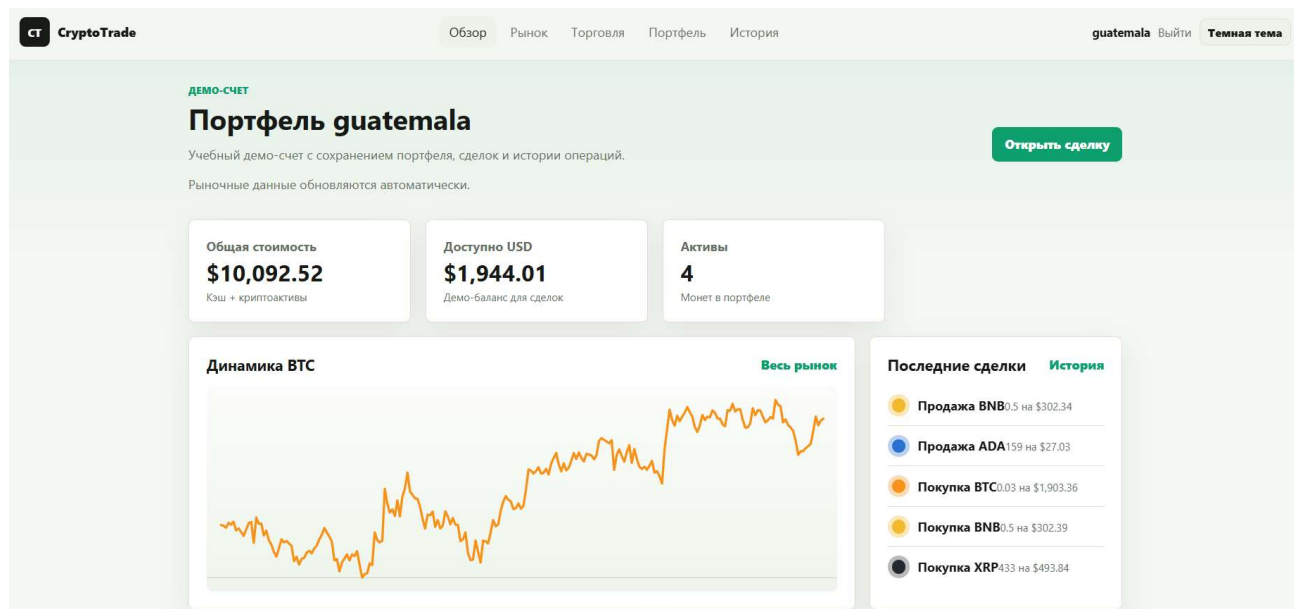


Рисунок 3.2 – Макет личной страницы пользователя

Страница market.html предназначена для просмотра рынка криптовалют, ее макет представлен на рисунке 3.3. В таблице отображаются символ, название, текущая цена, изменение, уровень риска, источник цены и ссылка на торговлю выбранным активом. На этой же странице выводится лента активности, где смешиваются реальные и симулируемые операции, что позволяет показать движение учебного рынка.

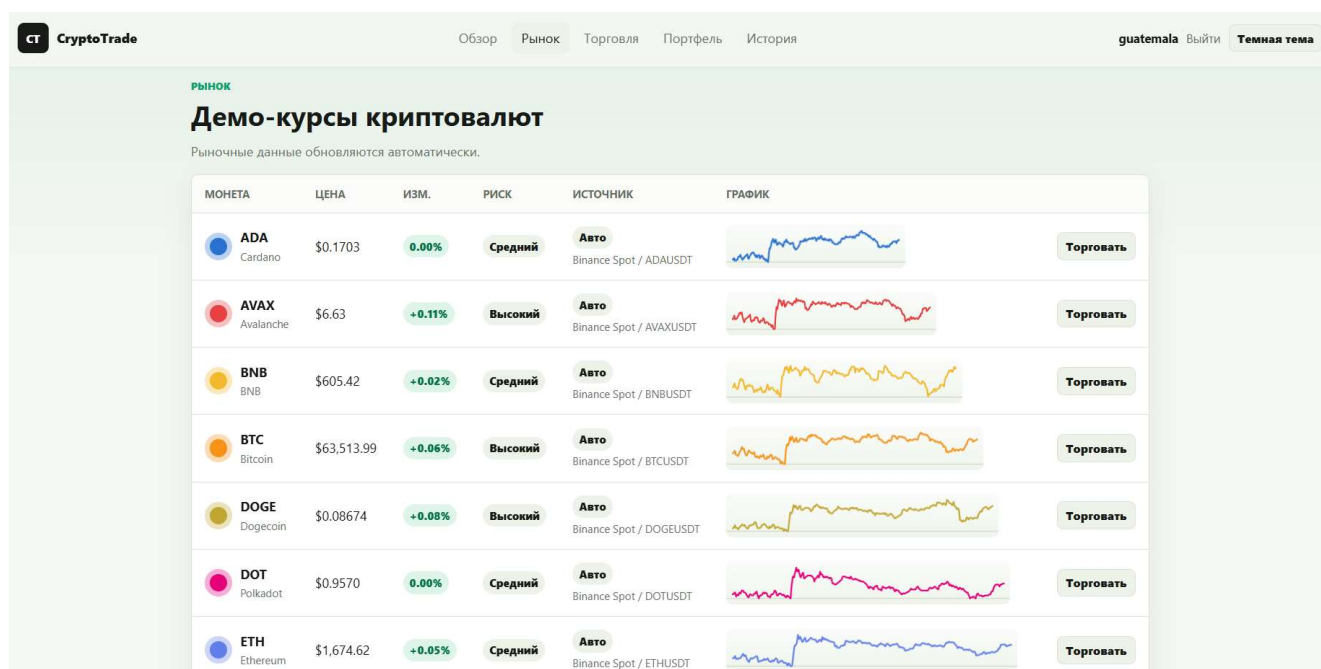


Рисунок 3.3 – Макет страницы рынка

Страница portfolio.html показывает состояние активов пользователя, ее макет представлен на рисунке 3.4. В верхней части выводятся общая стоимость, стоимость криптоактивов и свободные USD. Ниже формируются карточки монет, которые есть у

пользователя: количество, рыночная стоимость и доля в криптовалютной части портфеля. Если активов нет, выводится пустое состояние с подсказкой.

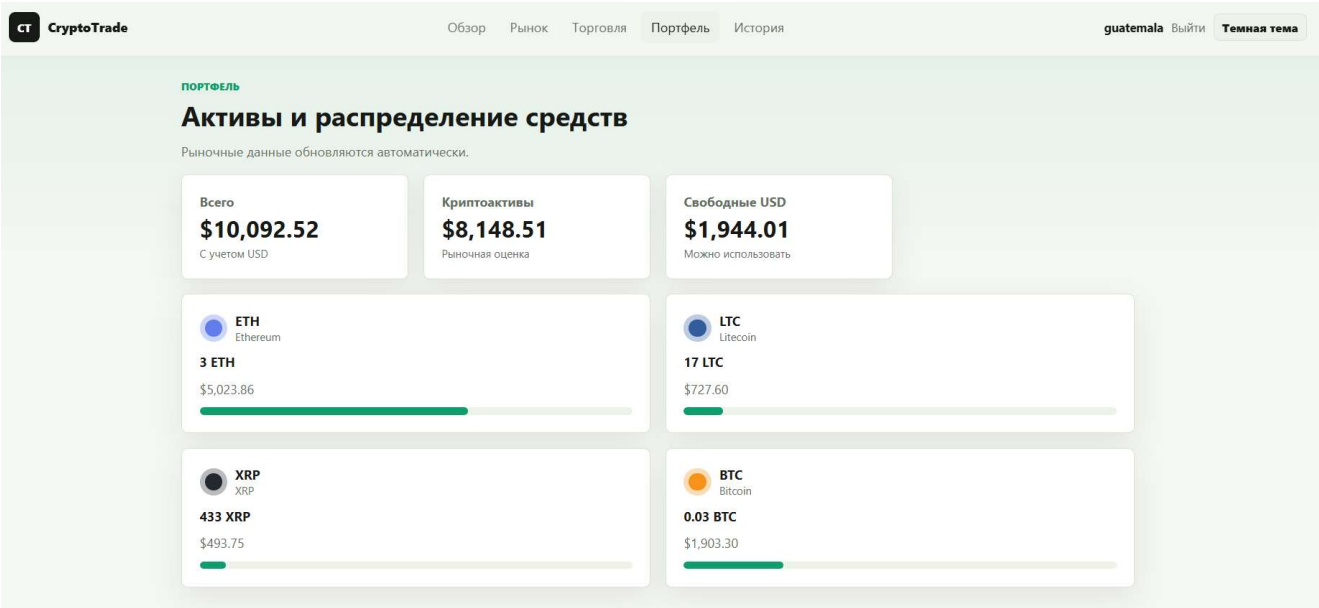


Рисунок 3.4 – Макет страницы портфеля

Страница history.html отображает личный журнал операций, ее макет представлен на рисунке 3.5. В таблице выводятся дата, тип операции, актив, количество, цена, сумма и статус. В отличие от административной панели, обычный пользователь видит только свои транзакции, что соответствует разграничению доступа по ролям.

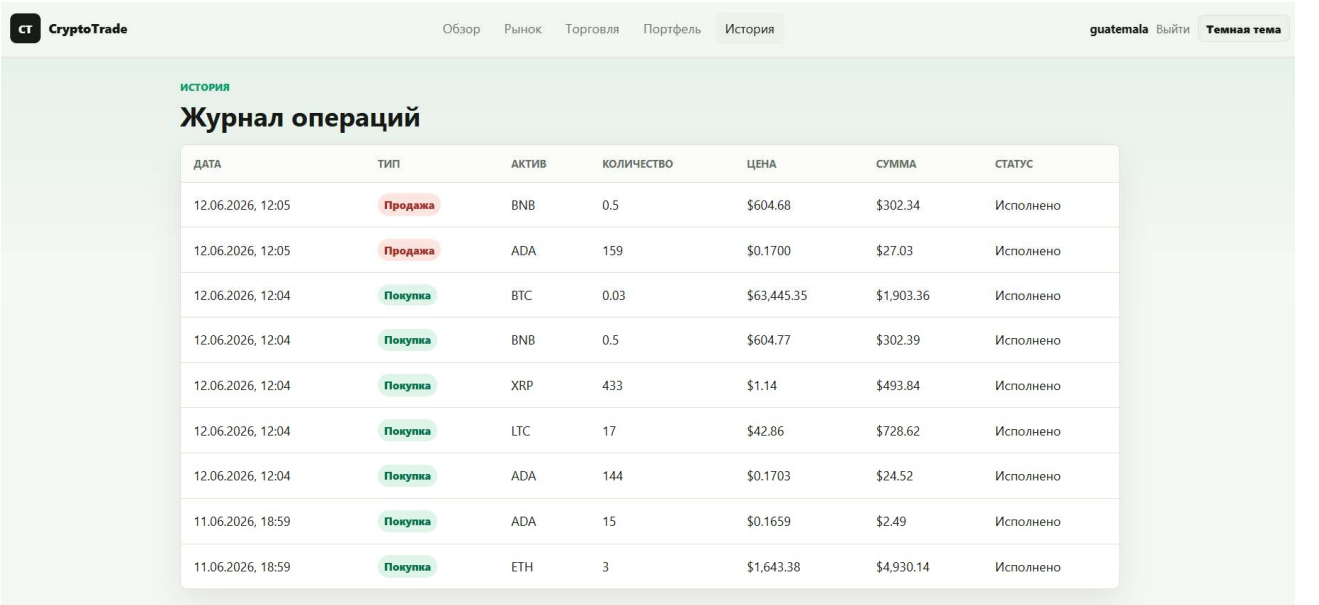


Рисунок 3.5 – Макет истории операций

3.5 Торговая страница и графики

Страница `trade.html` является основным рабочим экраном трейдера, ее макет представлен на рисунке 3.5. Пользователь выбирает криптовалюту, видит ее текущую цену, описание, уровень риска, доступный USD-баланс и количество выбранной монеты на кошельке. При вводе количества автоматически рассчитывается сумма операции.

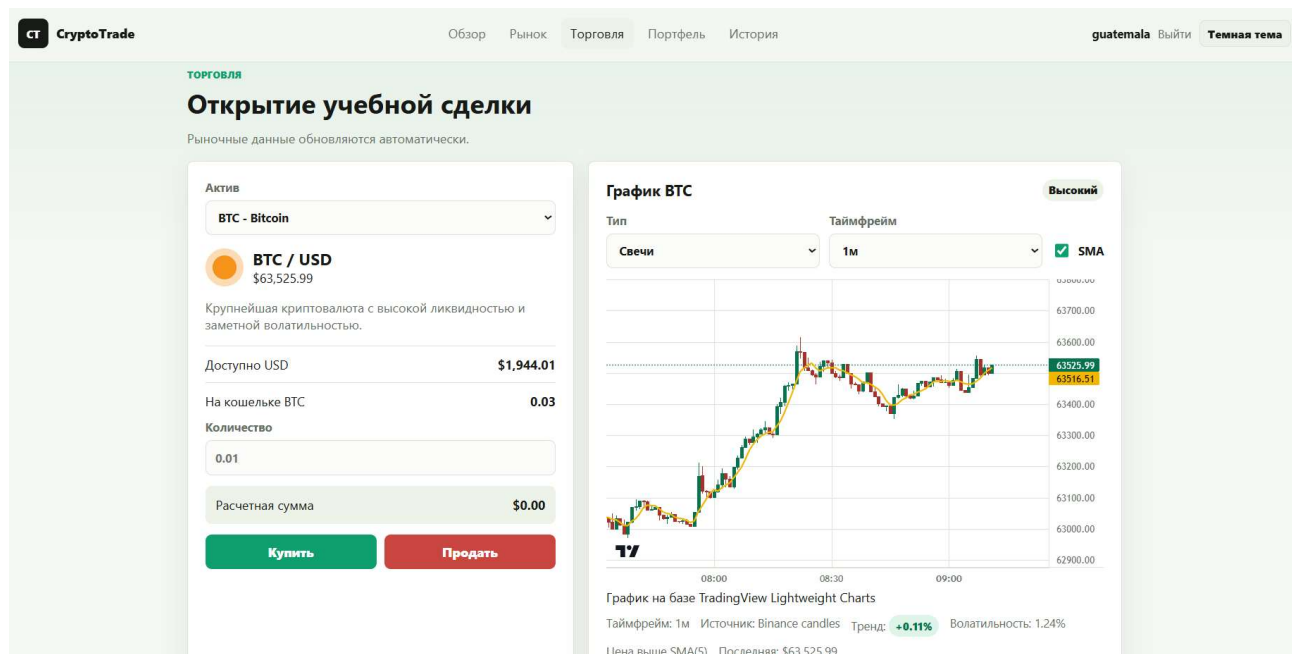


Рисунок 3.5 – Макет страницы торговли

Покупка и продажа выполняются одной формой. При отправке формы определяется тип операции: `buy` или `sell`. Функция `performTrade` проверяет наличие пользователя и актива, корректность количества, положительную сумму сделки, достаточность USD при покупке и достаточность монет при продаже. Если проверка не пройдена, пользователь получает понятное сообщение об ошибке.

При успешной покупке USD-баланс пользователя уменьшается, количество монет в кошельке увеличивается, а в таблицу `transactions` добавляется завершенная операция со статусом `Исполнено`. При продаже выполняется обратное изменение: количество монет уменьшается, USD-баланс увеличивается, а транзакция также сохраняется в истории. Отдельная таблица заявок не используется, потому что учебная сделка исполняется сразу.

На торговой странице размещен расширенный график. Пользователь может выбрать тип отображения: линия или японские свечи, а также таймфрейм 1м, 5м, 15м или 1ч. Дополнительно можно включить или выключить индикатор `SMA(5)`. Для построения графика используется `TradingView Lightweight Charts`, а если библиотека или внешние свечи недоступны, используется `canvas`-график и локальная история цен.

Изм.	Лист	№ докум.	Подпись	Дата

КП.AC576.220249 – 05 81 00

Лист

23

Свечные данные загружаются через API /api/klines. Backend сначала обращается к Binance Spot, а при ошибке использует Coinbase Exchange как запасной источник. Полученные свечи применяются не только для большого графика на странице торговли, но и для мини-графиков на страницах обзора и рынка. Это делает отображение динамики более приближенным к реальным торговым терминалам.

3.6 Клиентская логика JavaScript

Основная клиентская логика размещена в файле site/assets/js/app.js. Файл отвечает за нормализацию данных, чтение и сохранение состояния через API, авторизацию, регистрацию, расчеты портфеля, выполнение сделок, отображение страниц, построение графиков и работу административной панели.

В JavaScript-коде используется единая структура состояния: список пользователей и объект data, включающий currencies, wallets, transactions и settings. При загрузке страницы состояние получается из SQLite через backend.py, а после изменений отправляется обратно на сервер. Для удобства интерфейса поля преобразуются в camelCase, например balance_usd становится balanceUsd.

Отдельное внимание уделено производительности торговой страницы и страниц с графиками. Свечи кэшируются на клиенте, повторные запросы ограничиваются интервалом, а при изменении количества в торговой форме пересчитывается только сумма сделки без полной перерисовки страницы. Это уменьшает задержки при работе пользователя.

Основные функции JavaScript приведены в таблице 3.3.

Таблица 3.3 - Основные функции JavaScript

Функция	Назначение
initLoginForm	Обработка формы входа и переход в нужный раздел
initRegisterForm	Проверка регистрационных данных и создание нового трейдера
initSessionUi	Проверка сессии, прав доступа, навигации и выхода
performTrade	Проверка и выполнение покупки или продажи
renderDashboardPage	Отображение обзора портфеля и последних операций
renderMarketTable	Формирование таблицы криптовалют на странице рынка
initTradePage	Инициализация торговой формы и графика
fetchMarketCandlesAsync	Загрузка свечей через /api/klines и кэширование результата
drawTradingViewChart	Построение расширенного графика с линией, свечами и SMA

Продолжение таблицы 3.3

Функция	Назначение
renderPortfolioPage	Расчет и отображение портфеля пользователя
renderHistoryPage	Вывод личной истории операций
initAdminPage	Настройка форм админ-панели
syncMarketPrices	Обновление авто-курсов из Binance Spot
updateSimulationSettings	Сохранение ручных параметров симуляции
runSimulationTick	Генерация симулируемых сделок по заданной частоте

3.7 Backend/API и работа с SQLite

Серверная часть реализована в файле `site/backend.py` с использованием стандартных модулей Python: `http.server` для локального HTTP-сервера, `sqlite3` для работы с базой данных, `urllib` для обращения к внешним источникам свечей и `json` для обмена данными с клиентом.

При запуске `backend.py` вызывается `init_db`. Эта функция создает таблицы `roles`, `users`, `currencies`, `wallets`, `transactions`, `price_history` и `system_settings`, если они отсутствуют. Также добавляются демонстрационные роли, начальный набор криптовалют и служебная запись настроек симуляции.

Для совместимости с предыдущими версиями предусмотрена миграция `migrate_transactions_without_orders`. Если в старой базе присутствовала таблица `orders` или поле `order_id` в `transactions`, данные переносятся в новую структуру, где остается только таблица `transactions`. Это устраняет дублирование, потому что в учебной модели сделка выполняется сразу.

Маршрут `GET /api/state` возвращает полное состояние приложения: пользователей, валюты, кошельки, транзакции и настройки. Маршрут `POST /api/state` принимает измененное состояние и сохраняет его в SQLite. При сохранении используется не полная потеря старых данных, а добавление или обновление записей; отдельно удаляются только симулируемые записи, если они больше не присутствуют в состоянии.

Маршрут `GET /api/health` используется для простой проверки работоспособности сервера. Маршрут `GET /api/klines` получает свечи для выбранной торговой пары и таймфрейма. Если запрошен файл с расширением `.db`, сервер возвращает запрет, чтобы файл базы данных не отдавался как статический ресурс.

Основные API-маршруты приведены в таблице 3.4.

Таблица 3.4 - API backend.py

Маршрут	Метод	Назначение
/api/state	GET	Получение текущего состояния из SQLite
/api/state	POST	Сохранение пользователей, данных рынка, транзакций и настроек
/api/klines	GET	Получение свечей Binance Spot или Coinbase Exchange
/api/health	GET	Проверка запуска сервера и пути к базе данных
*.db	GET	Запрещенный доступ к файлу базы данных

3.8 Административная панель и симуляция

Страница admin.html доступна только пользователю с ролью admin. В верхней части панели отображаются четыре показателя: количество пользователей, количество завершенных сделок, общий оборот и состояние симуляции. Ниже расположены формы управления курсом, добавления монеты, настройки симуляции, журнал операций пользователей и таблица активов рынка.

Форма Курс и источник позволяет выбрать актив, изменить цену, указать режим цены и биржевую пару. В ручном режиме цена задается администратором и не перезаписывается автообновлением. В автоматическом режиме цена обновляется по публичной паре Binance Spot, например BTCUSDT. Для ручного запуска обновления предусмотрена кнопка Обновить авто-курсы.

Форма Добавить монету позволяет расширить список активов. Администратор вводит символ, название, начальную цену, уровень риска, описание, режим цены и при необходимости биржевую пару. После проверки данные добавляются в currencies, а новая монета появляется на рынке и в торговой форме.

Режим симуляции активности переделан на ручной ввод параметров. Администратор задает, сколько симулируемых пользователей должно существовать и сколько сделок в минуту нужно создавать. Симулируемые пользователи получают роль simulator, а связанные кошельки и транзакции помечаются признаком isSimulated.

Симуляция не догоняет пропущенное время после закрытия сервера. Если сайт или backend были выключены, после нового запуска генерация продолжается с текущего момента. При отключении симуляции удаляются только симулируемые пользователи, кошельки и транзакции, а реальные учетные записи и реальные сделки сохраняются.

Управление учебной биржей

Рыночные данные обновляются автоматически.

Пользователи

10

Всего аккаунтов

Сделки

286

Завершенные операции

Оборот

\$139,889.28

Демо-операции

Симуляция

Вкл.

6 пользователей, 15 сделок/мин, 272 операций

Курс и источник

Актив

ADA - Cardano

Режим цены

Авто из Binance

Биржевая пара для авто

ADAUSDT

Новая цена, USD

0.1703

Сохранить настройки

Обновить авто-курсы

Автообновление меняет только активы в режиме Binance. Ручные цены администратора остаются без изменений.

Добавить монету

Символ

XMR

Название

Monero

Начальная цена, USD

Режим цены

Ручной курс

Биржевая пара для авто

XMRUSDT

Уровень риска

Низкий

Описание

Краткое описание актива

Добавить актив

Симуляция активности

Вкл.

☒ Включить симуляцию пользователей и сделок

Симулируемых пользователей

6

Сделок в минуту

15

Сохранить режим

Добавить 1 сделку

Сделки создаются равномерно по времени. При выключении удаляются только записи с меткой симуляции: сгенерированные пользователи, операции и кошельки. Реальные регистрации и реальные сделки остаются.

Рисунок 3.6 – Макет панели администратора

3.9 Оформление, футер и документы

Визуальное оформление проекта находится в файле `site/assets/css/style.css`. Стиль сайта выполнен в темной теме, потому что такая цветовая схема привычна для финансовых и торговых интерфейсов. На страницах используются панели, таблицы, карточки показателей, компактные формы и цветовые метки риска.

Для адаптивности применяются гибкие сетки и ограничения ширины. Карточки рынка и портфеля перестраиваются в зависимости от ширины экрана, таблицы сохраняют читаемость, а формы остаются доступными на небольших экранах. Это важно, потому что пользователь может открыть учебный сайт как на ноутбуке, так и на мониторе с другой шириной. Футер присутствует на публичных и защищенных страницах. В нем указано название проекта CryptoTrade и размещены ссылки на [privacy.html](#) и [terms.html](#). Политика конфиденциальности объясняет, какие учебные данные используются и где они хранятся. Пользовательское соглашение фиксирует, что операции являются демонстрационными и не имеют финансовой силы.

Юридические страницы не требуют авторизации и доступны как до входа, так и после него. Это сделано специально: пользователь может ознакомиться с правилами до регистрации. Такие страницы не являются обязательными для простой учебной страницы, но делают проект более завершенным и похожим на реальный веб-сервис.

3.10 Алгоритмы работы программы

Основной алгоритм работы сайта начинается с запуска локального сервера. Backend создает или открывает SQLite-базу, проверяет наличие таблиц, добавляет начальные роли и активы, после чего начинает обслуживать HTTP-запросы. Пользователь открывает [index.html](#) и выполняет вход или регистрацию.

После авторизации клиент загружает состояние через API, отображает нужную страницу и разрешает пользователю выполнить действие. При покупке или продаже выполняется проверка средств, изменение баланса и кошелька, создание транзакции и сохранение изменений в SQLite. После сохранения интерфейс обновляет портфель, историю и последние операции.

Алгоритм обновления графиков включает выбор актива, определение торговой пары, запрос свечей через [/api/klines](#), нормализацию данных и построение графика. Если API возвращает ошибку или внешняя сеть недоступна, сайт использует локальные значения `price_history`, поэтому графический блок остается работоспособным.

Алгоритм симуляции использует настройки из `system_settings`. При включенной симуляции приложение рассчитывает, сколько сделок нужно создать за прошедший короткий интервал, генерирует операции равномерно и сохраняет дробный остаток в `simulation_carry`. При перезапуске сервера старое время не компенсируется, что предотвращает резкий выброс операций после простоя.

4. Тестирование

4.1 Методика тестирования

Тестирование сайта CryptoTrade выполнялось методом функциональной проверки пользовательских и административных сценариев. Основная цель тестирования состояла в том, чтобы убедиться, что сайт корректно выполняет вход, регистрацию, торговые операции, сохранение данных в SQLite, построение графиков, работу API и отделение симулируемых записей от реальных данных.

Проверка проводилась в локальной среде. Сайт запускался через start_cryptotrade.bat или командой python backend.py из папки site. После запуска открывался адрес <http://127.0.0.1:8000/index.html>. Такой способ запуска соответствует фактической архитектуре проекта, потому что HTML-страницы обращаются к Python API, а данные сохраняются в SQLite-базе site/cryptotrade.db.

Тестирование выполнялось вручную по контрольным сценариям. Для каждого сценария задавались начальные условия, выполнялось действие пользователя или администратора, затем проверялся ожидаемый результат на странице и в состоянии приложения. Отдельно проверялись ошибочные ситуации: неверный пароль, недостаточный USD-баланс, попытка продажи без монет и недоступность внешних свечных данных.

При тестировании использовались демонстрационные учетные записи: student@cryptotrade.local для трейдера и admin@cryptotrade.local для администратора. Также проверялась регистрация нового пользователя. Для административных сценариев использовалась роль admin, а для проверки симуляции - служебные пользователи с ролью simulator.

Тестовая среда приведена в таблице 4.1.

Таблица 4.1 - Тестовая среда

Параметр	Значение
Режим запуска	Локальный сервер backend.py или start_cryptotrade.bat
Адрес сайта	http://127.0.0.1:8000/index.html
База данных	site/cryptotrade.db
Основные технологии	HTML, CSS, JavaScript, Python, SQLite
Проверяемые API	/api/state, /api/klines, /api/health
Демо-трейдер	student@cryptotrade.local / student123
Демо-администратор	admin@cryptotrade.local / admin123

4.2 Направления проверки

Проверка авторизации и регистрации была направлена на подтверждение того, что пользователь может войти в систему, зарегистрировать новый аккаунт, выйти из системы и не получить доступ к защищенным страницам без сессии. Дополнительно проверялось, что обычный трейдер не может открыть административную панель.

Проверка торговых операций включала покупку и продажу активов. Для покупки контролировалось уменьшение USD-баланса и увеличение количества монет в кошельке. Для продажи проверялось обратное изменение: уменьшение количества монет и увеличение USD-баланса. В обоих случаях проверялось создание записи в transactions.

Проверка хранения данных была связана с SQLite. После регистрации, покупки или продажи выполнялся перезапуск backend.py, затем сайт открывался повторно. Если данные пользователя, кошелек и истории операций сохранялись, сценарий считался выполненным.

Проверка графиков включала отображение мини-графиков на страницах обзора и рынка, а также расширенного графика на странице торговли. Проверялись переключение типа графика, таймфрейма и индикатора SMA. Также проверялось, что при недоступности внешних свечей сайт использует fallback-историю из price_history.

Проверка административной панели включала просмотр статистики, обновление ручного курса, настройку автоматического режима цены, добавление новой монеты, запуск обновления курсов, просмотр операций всех пользователей и настройку симуляции активности.

Проверка симуляции была выделена отдельно, потому что симулируемые данные не должны повреждать реальные данные. Проверялось создание пользователей с ролью simulator, генерация заданного количества сделок в минуту, отсутствие догоняющей генерации после перезапуска сервера и удаление только записей с признаком isSimulated при отключении симуляции.

4.3 Тестовые сценарии

В рамках тестирования были проверены основные сценарии работы сайта CryptoTrade: регистрация и авторизация пользователя, просмотр рынка криптовалют, выбор актива, получение графика, выполнение операций покупки и продажи, а также отображение портфеля и истории транзакций. Отдельно проверялись ошибочные ситуации: ввод некорректных данных, попытка покупки при недостаточном USD-

					КП.АС576.220249 – 05 81 00	Лист
						30
Изм.	Лист	№ докум.	Подпись	Дата		

балансе, попытка продажи при недостаточном количестве монет и недоступность внешнего источника свечей с переходом на локальную fallback-историю.

Результаты тестирования приведены в таблице 4.2.

Таблица 4.2 - Тестирование сайта

№	Проверка	Действие	Ожидаемый результат	Статус
1	Запуск сервера	Запустить start_cryptotrade.bat или python backend.py	Открывается локальный сервер, доступен index.html	Выполнено
2	Проверка /api/health	Открыть /api/health	Возвращается JSON с ok=true и путем к базе данных	Выполнено
3	Вход трейдера	Ввести student@cryptotrade.local и student123	Открывается dashboard.html	Выполнено
4	Вход администратора	Ввести admin@cryptotrade.local и admin123	Открывается admin.html	Выполнено
5	Неверный пароль	Ввести неправильный пароль	Появляется сообщение об ошибке	Выполнено
6	Регистрация пользователя	Создать новый аккаунт на index.html	Создается трейдер с балансом 10000 USD	Выполнено
7	Защита админ-панели	Открыть admin.html под трейдером	Пользователь перенаправляется на dashboard.html	Выполнено
8	Загрузка состояния	Открыть страницу после входа	Данные пользователей, валют и кошельков загружаются через /api/state	Выполнено
9	Покупка при достаточном балансе	Купить доступное количество BTC или другой монеты	USD уменьшается, кошелек увеличивается, создается transaction	Выполнено
10	Покупка при недостаточном балансе	Указать слишком большое количество монет	Операция не выполняется, выводится ошибка	Выполнено
11	Продажа при наличии монет	Продать часть купленного актива	Количество монет уменьшается, USD увеличивается	Выполнено
12	Продажа без монет	Попробовать продать актив, которого нет в кошельке	Операция блокируется, выводится ошибка	Выполнено
13	История пользо-	Открыть history.html по-	В таблице отображается	Выполнено

Изм.	Лист	№ докум.	Подпись	Дата

КП.АС576.220249 – 05 81 00

Лист

31

№	Проверка	Действие	Ожидаемый результат	Статус
	вателя	сле сделки	личная операция пользователя	
14	Портфель	Открыть portfolio.html после покупки	Отображаются активы, стоимость и доли портфеля	Выполнено
15	Перезапуск backend.py	Закрыть и снова запустить сервер	Пользователи, кошельки и транзакции сохраняются в cryptotrade.db	Выполнено
16	Ручное изменение курса	В admin.html изменить цену выбранного актива	Цена обновляется, история цен пополняется	Выполнено
17	Добавление монеты	Заполнить форму добавления актива	Новая монета появляется на рынке и в торговой форме	Выполнено
18	Автообновление курсов	Включить авто-режим и нажать обновление	Цена обновляется по Binance Spot или выводится понятное сообщение	Выполнено
19	Расширенный график	Открыть trade.html и переключить тип графика	Отображаются линия или японские свечи	Выполнено
20	Таймфрейм и SMA	Сменить таймфрейм и выключить/включить SMA	График перестраивается без перезагрузки страницы	Выполнено
21	Fallback-график	Открыть график при недоступных внешних свечах	Используется локальная история price_history	Выполнено
22	Настройка симуляции	Задать пользователей и сделки в минуту	Создаются пользователи simulator и симулируемые сделки	Выполнено
23	Нет догоняющей симуляции	Перезапустить сервер после простоя	Сделки не создаются пачкой за время простоя	Выполнено
24	Отключение симуляции	Выключить симуляцию в admin.html	Удаляются только isSimulated-записи, реальные данные сохраняются	Выполнено
25	Политика и соглашение	Открыть privacy.html и terms.html из футера	Страницы доступны без авторизации	Выполнено
26	Запрет доступа к базе	Попробовать открыть файл .db через сервер	Сервер возвращает запрет доступа к базе как к статическому файлу	Выполнено

4.4 Анализ результатов тестирования

По результатам проверки основные пользовательские сценарии выполнены успешно. Трейдер может зарегистрироваться, войти в систему, просматривать рынок, выполнять покупку и продажу, видеть портфель и историю операций. Ошибочные сценарии также обрабатываются корректно: при неверном пароле, недостаточном USD-балансе или нехватке монет система не изменяет данные и выводит сообщение.

Проверка SQLite подтвердила, что данные сохраняются между запусками backend.py. После перезапуска сервера пользователи, кошельки, транзакции, активы и настройки симуляции остаются доступными. Это подтверждает переход проекта от хранения данных в браузере к полноценной файловой базе данных.

Проверка API показала, что клиентская часть получает состояние через /api/state, сохраняет изменения POST-запросом на /api/state и использует /api/klines для получения свечей. При отсутствии внешних данных интерфейс продолжает работать за счет fallback-истории, поэтому отказ внешнего источника не блокирует торговые страницы.

Проверка административной панели подтвердила возможность управлять активами, обновлять курсы, добавлять монеты и просматривать операции всех пользователей. Режим симуляции работает отдельно от реальных данных: записи получают признак isSimulated и могут быть удалены без потери реальных пользователей и транзакций.

В результате тестирования критических ошибок, мешающих выполнению основных функций курсового проекта, не обнаружено. Оставшиеся ограничения связаны с учебным характером системы: пароли не хешируются, внешние рыночные данные зависят от доступности публичных API, а SQLite используется как локальная файловая база данных, а не промышленная серверная СУБД.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсового проекта была достигнута поставленная цель: разработан сайт CryptoTrade для учебной симуляции торговли криптовалютой. Сайт позволяет пользователю зарегистрироваться, войти в систему, просматривать рынок активов, выполнять учебные операции покупки и продажи, анализировать портфель и просматривать историю сделок.

В ходе работы был выполнен анализ предметной области и определены основные роли системы: трейдер, администратор и служебная роль simulator для данных, созданных режимом симуляции. Были описаны основные бизнес-процессы: регистрация, авторизация, просмотр рынка, выполнение сделки, просмотр портфеля, просмотр истории, управление курсами, добавление активов и симуляция активности.

Для проекта разработана логическая и физическая модель базы данных. Реализация выполнена на SQLite в файле cryptotrade.db. В базе используются связанные таблицы roles, users, currencies, wallets, transactions, price_history и system_settings. Такая структура соответствует требованию о наличии не менее пяти связанных сущностей и позволяет хранить пользователей, активы, кошельки, операции, историю цен и настройки симуляции.

Проект был переведен с хранения данных в localStorage на локальный Python backend/API и SQLite-базу. Данные пользователей, кошельков, транзакций, цен и настроек симуляции сохраняются между запусками сервера. Отдельная таблица orders не используется, так как учебные сделки исполняются сразу и фиксируются в transactions.

Пользовательская часть сайта включает страницы входа и регистрации, обзора, рынка, торговли, портфеля и истории операций. На странице торговли реализована форма покупки и продажи с проверкой достаточности USD или монет. После успешной операции обновляется баланс, изменяется кошелек и создается запись транзакции.

В проект добавлены расширенные графики на базе TradingView Lightweight Charts. Графики поддерживают линию и японские свечи, несколько таймфреймов и индикатор SMA(5). Свечные данные загружаются через API из открытых источников Binance Spot и Coinbase Exchange, а при недоступности сети используется локальная fallback-история цен.

Административная часть предоставляет функции контроля учебной биржи: просмотр статистики, общий журнал операций пользователей, список активов, ручное изменение цены, автоматическое обновление курсов, добавление новых монет и управление симуляцией активности. Симуляция настраивается вручную по количеству пользователей и числу сделок в минуту, а симулируемые записи отделяются от реальных признаком isSimulated.

					КП.АС576.220249 – 05 81 00	Лист
						34
Изм.	Лист	№ докум.	Подпись	Дата		

Дополнительно были добавлены футер, политика конфиденциальности и пользовательское соглашение. Эти страницы фиксируют учебный характер проекта, отсутствие реальных финансовых операций и особенности локального хранения данных. Темная тема и устранение белой вспышки при загрузке улучшают визуальное восприятие интерфейса.

Проведенное тестирование подтвердило работоспособность основных функций сайта: авторизации, регистрации, торговли, портфеля, истории, административной панели, SQLite-хранения, API, графиков, fallback-истории, симуляции и страниц документов. Ошибочные сценарии, такие как неверный пароль, недостаточный баланс и попытка продажи отсутствующих монет, обрабатываются корректно.

Разработанный проект соответствует требованиям курсовой работы: содержит сайт и пояснительную записку, авторизацию и регистрацию, связанную модель данных, базу данных, административную часть, блок-схему, схему базы данных и результаты тестирования. Проект может быть развит дальше за счет хеширования паролей, перехода на серверную СУБД, добавления миграций, резервного копирования и более строгой серверной авторизации.

СПИСОК СОКРАЩЕНИЙ

БД - база данных.

ПО - программное обеспечение.

ПЗ - пояснительная записка.

ИС - информационная система.

СУБД - система управления базами данных.

SQL - язык структурированных запросов.

HTML - язык гипертекстовой разметки.

CSS - каскадные таблицы стилей.

JS - JavaScript.

DOM - объектная модель документа.

UI - пользовательский интерфейс.

UX - пользовательский опыт.

API - программный интерфейс приложения.

HTTP - протокол передачи гипертекста.

URL - унифицированный указатель ресурса.

JSON - текстовый формат обмена структурированными данными.

CRUD - операции создания, чтения, изменения и удаления данных.

GET - HTTP-метод получения данных.

POST - HTTP-метод отправки данных.

REST - архитектурный стиль построения веб-API.

Backend - серверная часть приложения.

Frontend - клиентская часть приложения.

SQLite - файловая реляционная система управления базами данных.

DB - database, база данных.

CSV - текстовый формат табличных данных с разделителями.

СПИСОК ИСТОЧНИКОВ

1. ГОСТ 2.105-95. Единая система конструкторской документации. Общие требования к текстовым документам.
2. ГОСТ 19.101-77. Единая система программной документации. Виды программ и программных документов.
3. ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения.
4. Binance Open Platform. Spot API Market Data Endpoints [Электронный ресурс]. URL: <https://developers.binance.com/docs/binance-spot-api-docs/rest-api/market-data-endpoints> (дата обращения: 11.06.2026).
5. Coinbase Developer Documentation. Get product candles [Электронный ресурс]. URL: <https://docs.cdp.coinbase.com/api-reference/exchange-api/rest-api/products/get-product-candles> (дата обращения: 11.06.2026).
6. MDN Web Docs. Canvas API [Электронный ресурс]. URL: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API (дата обращения: 11.06.2026).
7. MDN Web Docs. CSS: Cascading Style Sheets [Электронный ресурс]. URL: <https://developer.mozilla.org/en-US/docs/Web/CSS> (дата обращения: 11.06.2026).
8. MDN Web Docs. Fetch API [Электронный ресурс]. URL: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API (дата обращения: 11.06.2026).
9. MDN Web Docs. HTML: HyperText Markup Language [Электронный ресурс]. URL: <https://developer.mozilla.org/en-US/docs/Web/HTML> (дата обращения: 11.06.2026).
10. MDN Web Docs. JavaScript [Электронный ресурс]. URL: <https://developer.mozilla.org/en-US/docs/Web/JavaScript> (дата обращения: 11.06.2026).
11. MDN Web Docs. Window: localStorage property [Электронный ресурс]. URL: <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage> (дата обращения: 11.06.2026).
12. Python Documentation. http.server - HTTP servers [Электронный ресурс]. URL: <https://docs.python.org/3/library/http.server.html> (дата обращения: 11.06.2026).
13. Python Documentation. json - JSON encoder and decoder [Электронный ресурс]. URL: <https://docs.python.org/3/library/json.html> (дата обращения: 11.06.2026).
14. Python Documentation. sqlite3 - DB-API 2.0 interface for SQLite databases [Электронный ресурс]. URL: <https://docs.python.org/3/library/sqlite3.html> (дата обращения: 11.06.2026).